

Week4 예습과제

CHAPTER 4 PART 2

▼ PART 5 GBM (Gradient Boosting Machine)

- 에이다부스트와 유사하나 가중치 업데이트를 경사 하강법으로 수행 — 오류식 $h(x) = y - F(x)$ 최소화 방향으로 반복 업데이트
- 분류·회귀 모두 가능 — 사이킷런 GradientBoostingClassifier / Regressor
- 실습(사용자 행동 인식 데이터): 정확도 93.89%로 랜덤 포레스트보다 우수하나 수행 시간 약 537초로 매우 오래 걸림
- 순차적 오류 보정 학습이라 병렬 처리 불가 — 대용량 데이터 학습 시간이 GBM의 핵심 과제

▼ 하이퍼 파라미터

- loss: 비용 함수, 기본값 'deviance'
- learning_rate: 학습률(0~1, 기본 0.1) — 작으면 정교하지만 느림, 크면 빠르지만 성능 저하 우려
- n_estimators: weak learner 개수(기본 100)
- subsample: 학습 데이터 샘플링 비율(기본 1)
- learning_rate와 n_estimators는 상호 보완 관계로 함께 조합

▼ PART 6 XGBoost (eXtra Gradient Boost)

- 트리 기반 앙상블 중 가장 각광받는 알고리즘 — 캐글 경연 대회에서 널리 사용되며 유명해짐
- GBM 기반이나 병렬 학습으로 느린 속도·과적합 규제 부재 문제를 개선
- 장점: 뛰어난 예측 성능 / GBM 대비 빠른 수행 시간 / 자체 과적합 규제 / Tree pruning / 자체 교차 검증(조기 중단) / 결손값 자체 처리
- 파이썬 네이티브 xgboost 패키지 + 사이킷런 래퍼(XGBClassifier·XGBRegressor) — 실무는 GridSearchCV·Pipeline과 호환되는 사이킷런 래퍼를 주로 사용

▼ 파이썬 네이티브 XGBoost

- 전용 데이터 객체 DMatrix 사용(data=피쳐, label=레이블)
- train()에 파라미터·DMatrix·num_boost_round·early_stopping_rounds·evals를 전달해 학습
- predict()는 0/1이 아닌 예측 확률을 반환 — 임계값(예: 0.5) 기준으로 레이블 결정 필요

▼ 주요 하이퍼 파라미터

- booster: gbtrees / gblinear
- eta(=learning_rate): 기본 0.3(사이킷런 래퍼는 0.1) — 보통 0.01~0.2 사용
- num_boost_round(=n_estimators)
- min_child_weight: 분할에 필요한 데이터 가중치 합, 클수록 과적합 억제

- gamma(=min_split_loss): 리프 추가 분할 최소 손실 감소값, 클수록 과적합 억제
- max_depth: 기본 6, 보통 3~10
- subsample / colsample_bytree: 샘플·피쳐 샘플링 비율(과적합 제어)
- lambda(=reg_lambda) / alpha(=reg_alpha): L2 / L1 규제
- objective / eval_metric: 손실 함수(binary:logistic 등) / 평가지표(logloss, error, auc 등)
- 사이킷런 래퍼는 이름 변경:
eta → learning_rate, sub_sample → subsample, lambda → reg_lambda, alpha → reg_alpha

▼ 조기 중단(Early Stopping)

- n_estimators에 도달하기 전이라도 성능이 더 개선되지 않으면 학습을 중지해 시간 단축
- early_stopping_rounds + eval_set + eval_metric을 함께 지정 — 값을 너무 작게 주면 충분히 학습되기 전에 멈춰 성능이 떨어질 수 있음

▼ PART 7 LightGBM

- XGBoost보다 학습 시간·메모리 사용량이 훨씬 적음 — 예측 성능은 XGBoost와 비슷
- 단점: 적은 데이터(대략 1만 건 이하)에서는 과적합이 발생하기 쉬움
- 균형 트리 분할(Level Wise) 대신 리프 중심 트리 분할(Leaf Wise) 사용 — 최대 손실값을 가진 리프를 지속 분할해 깊고 비대칭적인 트리 생성, 반복할수록 예측 오류 손실을 더 최소화
- 장점: 빠른 학습·예측, 적은 메모리, 카테고리형 피쳐 자동 최적 분할(원-핫 인코딩 불필요), GPU 지원
- lightgbm 패키지 — 사이킷런 래퍼 LGBMClassifier / LGBMRegressor 제공

▼ 주요 하이퍼 파라미터

- num_iterations(=n_estimators, 기본 100)
- learning_rate(기본 0.1)
- max_depth(기본 -1, 무제한) — Leaf Wise 특성상 상대적으로 깊어짐
- min_data_in_leaf(=min_child_samples): 리프가 되기 위한 최소 레코드 수
- num_leaves(기본 31): 트리 최대 리프 개수 — 모델 복잡도를 제어하는 핵심 파라미터
- boosting: gbdt / rf
- bagging_fraction(=subsample) / feature_fraction(=colsample_bytree)
- lambda_l2(=reg_lambda) / lambda_l1(=reg_alpha)
- 튜닝 방안: num_leaves를 중심으로 min_data_in_leaf·max_depth를 함께 조정해 복잡도를 낮추는 것이 기본 — learning_rate↓ + n_estimators↑ 조합도 기본적인 부스팅 튜닝 방안 (단, n_estimators 과도하면 과적합 우려)
- 실습(위스콘신 유방암 데이터): 조기 중단 적용 가능, 정확도 약 95.61%로 XGBoost와 유사

▼ PART 8 베이지안 최적화 기반 HyperOpt 하이퍼 파라미터 튜닝

- Grid Search는 하이퍼 파라미터 개수·범위가 넓거나 데이터가 대용량이면 최적화 시간이 기하급수적으로 늘어남 — XGBoost·LightGBM처럼 파라미터가 많은 알고리즘일수록 문제가 커짐
- 베이지안 최적화: 목적 함수를 모르는 블랙박스 상황에서 최소한의 시도로 최적 입력값을 빠르게 찾는 기법
- 구성 요소: 대체 모델(Surrogate Model, 관측값 기반 최적 함수 추정) + 획득 함수(Acquisition Function, 다음 관측 지점 계산)
- 절차: 랜덤 샘플링·관측 → 대체 모델이 최적 함수 추정 → 획득 함수가 다음 관측 지점 계산 → 재관측·모델 갱신을 반복하며 점차 정확해짐
- 대체 모델은 보통 가우시안 프로세스를 쓰지만, HyperOpt는 TPE(Tree-structured Parzen Estimator) 사용

▼ HyperOpt 사용법

- ① hp 모듈로 검색 공간(Search Space) 설정
- ② 목적 함수(Objective Function) 정의 — 검색 공간 딕셔너리를 받아 값 또는 {'loss', 'status'} 반환
- ③ fmin(fn, space, algo, max_evals, trials)으로 반환값이 최소가 되는 최적 입력값 탐색

▼ 유의 사항

- 검색 공간 값은 모두 실수형 — 정수형 하이퍼 파라미터(예: max_depth)는 명시적 형변환 필요
- HyperOpt는 최솟값을 찾는 방식 — 정확도처럼 클수록 좋은 지표는 -1을 곱해 반환해야 함 (MAE·RMSE 등은 그대로)
- cross_val_score()는 XGBoost·LightGBM의 조기 중단을 지원하지 않음 — 조기 중단이 필요하다면 KFold로 직접 교차 검증 구현
- 실습(위스콘신 유방암 데이터, XGBoost): 검색 공간을 설정해 fmin()으로 최적 파라미터 도출 후 재학습 — 튜닝 전보다 소폭 개선(데이터가 작아 rstate에 따라 결과가 불안정할 수 있음)

▼ PART 11 스택킹 앙상블

- 개별 알고리즘의 예측 결과를 다시 학습 데이터로 만들어 별도의 메타 모델로 최종 학습·예측 — 배깅·부스팅과의 차이점
- 필요 모델: 개별 기반 모델(여러 개) + 메타 모델
- 현실 모델보다는 캐글 등 대회에서 성능을 조금이라도 높이기 위해 사용 — 2~3개로는 효과가 작고, 스택킹이 성능 향상을 보장하지도 않음(비슷한 성능의 모델들을 결합하는 것이 일반적)

▼ 기본 스택킹

- 개별 모델들의 테스트 예측값(레이블)을 칼럼으로 결합해 메타 모델의 학습 피처로 사용
- 문제점: 테스트 데이터의 예측 결과로 메타 모델을 학습해 과적합이 발생하기 쉬움

▼ CV 세트 기반 스택킹

- 교차 검증으로 메타 모델용 학습·테스트 데이터를 생성해 과적합을 개선

- 스텝 1(개별 모델별): 학습 데이터를 N개 폴드로 나눠 반복 — 검증 폴드 예측값을 모아 메타 모델 학습 데이터로, 원본 테스트 데이터에 대한 예측을 폴드마다 구해 평균한 값을 메타 모델 테스트 데이터로 사용
- 스텝 2: 각 모델이 만든 학습·테스트 데이터를 모두 합쳐(concatenate) 메타 모델의 최종 학습·테스트 데이터 세트 생성 → 메타 모델 학습 후 최종 예측·평가
- 실습(위스콘신 유방암 데이터): 기본 스택킹 정확도 약 97.37% → CV 세트 기반 스택킹 약 98.25%로 개선
- 스택킹 튜닝은 보통 개별 알고리즘 각각을 최적화하는 것을 의미 — 분류뿐 아니라 회귀에도 적용 가능